

Eco Tracker

Wie nachhaltig lebst du?

Projektdokumentation – WMC 2. Semester

Dominic Schmidinger

1. Projektbeschreibung

Die Webanwendung **Eco Tracker** ist ein interaktives Projekt zum Thema Umwelt und Nachhaltigkeit. Ziel der Anwendung ist es, Nutzer auf spielerische Weise für ihr eigenes Verhalten im Alltag zu sensibilisieren und gleichzeitig reale Umweltdaten darzustellen.

Die Webseite besteht aus vier Seiten: **Login** (Einstieg), **Startseite** (Projektbeschreibung), **Quiz** (interaktiver Fragebogen) und **Ergebnisse** (Score + Live-Umweltdaten).

2. Verwendete Technologien

HTML5 – Struktur der vier Seiten (index.html, quiz.html, ergebnisse.html, login.html)

CSS3 – Design, Layout, Responsive Design mit Media Queries

JavaScript – Quiz-Logik, DOM-Manipulation, fetch()-Abfragen, Local Storage

Nominatim API – Geocoding (Stadtname zu Koordinaten)

Open-Meteo API – Wetterdaten und Solarstrahlung

Open-Meteo Air Quality API – Luftqualitätsdaten (European AQI)

fragen.json – Externe JSON-Datei für die Quiz-Fragen

3. Seitenstruktur und Navigation

Die Navigationsleiste ist auf allen Seiten (außer Login) fixiert und ermöglicht den Wechsel zwischen Startseite, Quiz und Ergebnisse. Auf der Startseite gibt es zusätzlich einen fixierten **Quiz starten**-Button, der beim Scrollen immer sichtbar bleibt.

4. Erklärung der wichtigsten Code-Passagen

4.1 Fragen aus JSON-Datei laden

Die Quiz-Fragen werden nicht direkt im JavaScript-Code gespeichert, sondern aus einer externen Datei **fragen.json** geladen. Das hat den Vorteil, dass Fragen leicht ergänzt werden können ohne den Code zu ändern.

```
fetch("fragen.json") .then(response => response.json()) .then(data => {
  questions = data; displayQuestion(); // erst nach dem Laden anzeigen });
```

fetch() lädt die Datei asynchron. Da das Laden Zeit braucht, arbeitet JavaScript mit **.then()** – der Code darin wird erst ausgeführt wenn die Daten vollständig angekommen sind.

4.2 Local Storage

Der Quiz-Score wird nach Abschluss des Quiz im Local Storage des Browsers gespeichert. So kann die Ergebnisseite den Wert auslesen, obwohl sie eine eigene Seite ist.

```
// Score speichern (quiz.html) localStorage.setItem("ecoQuizScore", finalScore);  
// Score auslesen (ergebnisse.html) const savedScore =  
localStorage.getItem("ecoQuizScore") || 0;
```

4.3 Globale Variablen für den Filter

Die Solarstrahlungsdaten werden in globalen Arrays gespeichert. Dadurch kann der Filter-Button auf die Daten zugreifen, auch nachdem der **fetch()** bereits abgeschlossen ist.

```
let radiationData = []; // Strahlungswerte (z.B. [0, 0, 30.8, 141, ...])  
let timesData = []; // Uhrzeiten (z.B. ["2026-06-06T00:00", ...])  
// Im fetch-block befüllen: radiationData = data.hourly.shortwave_radiation; timesData =  
data.hourly.time;
```

4.4 Die drei fetch()-Abfragen

Fetch 1 – Geocoding (Nominatim API):

Wandelt den eingegebenen Stadtnamen in GPS-Koordinaten (Latitude/Longitude) um. Diese Koordinaten werden für die weiteren API-Abfragen benötigt.

```
fetch(`https://nominatim.openstreetmap.org/search?q=${city}&format=json&limit;=1`)
```

Fetch 2 – Wetter + Solarstrahlung (Open-Meteo API):

Lädt aktuelle Wetterdaten (Temperatur, Wind) sowie stündliche Solarstrahlungswerte für 24 Stunden. Die Solarstrahlung wird als Array ausgegeben und mit einer Schleife als Liste angezeigt.

```
fetch(`https://api.open-meteo.com/v1/forecast?latitude=${lat}  
&longitude=${lon}&current_weather=true&hourly=shortwave_radiation`)
```

Fetch 3 – Luftqualität (Open-Meteo Air Quality API):

Lädt den aktuellen European AQI (Air Quality Index) für die eingegebene Stadt. Der Wert wird mit einer Hilfsfunktion in eine verständliche Beschreibung umgewandelt (z.B. 'Mäßig').

```
fetch(`https://air-quality-api.open-meteo.com/v1/air-quality?latitude=${lat}  
&longitude=${lon}&current=european_aqi`)
```

4.5 Array mit Schleife als Liste ausgeben

Die 24 Solarstrahlungswerte werden mit einer for-Schleife durchlaufen und als HTML-Cards ausgegeben. Dabei wird mit **split('T')** nur die Uhrzeit aus dem Datums-String

extrahiert.

```
for (let i = 0; i < 24; i++) { const hour = timesData[i].split("T")[1]; //  
"2026-06-06T08:00" → "08:00" listHTML += ` <div class="radiation-card">  
<span>${hour}</span> <span>${radiationData[i]} W/m2</span> </div> `; }
```

4.6 Filter-Funktion

Der Filter-Button liest den eingegebenen Mindestwert aus und zeigt nur jene Stunden an, bei denen die Solarstrahlung den Mindestwert erreicht oder überschreitet. Die Anzeige wird direkt ersetzt – nicht neu hinzugefügt.

```
filterBtn.onclick = () => { const minValue =  
parseInt(document.getElementById("filter-input").value); let filterHTML = "";  
for (let i = 0; i < 24; i++) { if (radiationData[i] >= minValue) { // nur Werte  
ueber Minimum filterHTML += `<div class="radiation-card">...</div>`; } }  
document.getElementById("radiation-list").innerHTML = filterHTML; };
```

4.7 Responsive Design

Mit einem CSS Media Query wird das Layout für mobile Geräte (unter 600px Breite) angepasst. Die Navigationsleiste wechselt von horizontal auf vertikal, und der Inhaltsbereich bekommt weniger Padding.

```
@media (max-width: 600px) { .navbar { flex-direction: column; } .nav-links {  
flex-direction: column; } }
```

5. Erfüllte Beurteilungskriterien

- ✓ **Punkte Abgabe:** Projekt vollständig abgegeben
- ✓ **Responsive Design:** Media Query fuer mobile Geraete (max-width: 600px)
- ✓ **Navigation:** Fixierte Navigationsleiste auf allen Seiten
- ✓ **3 fetch-Abfragen:** Geocoding, Wetter/Solar, Luftqualitaet
- ✓ **Array als Liste:** 24 Solarstrahlungswerte werden per Schleife ausgegeben
- ✓ **Filter:** Solarstrahlung nach Mindestwert filterbar
- ✓ **Local Storage:** Quiz-Score wird gespeichert und auf Ergebnisseite ausgelesen
- ✓ **JSON-File:** Quiz-Fragen werden aus fragen.json geladen
- ✓ **Dokumentation:** Dieses Dokument